WEEK #8

1# A file management system needs to organize a batch of filenames based on the length of the name rather than alphabetical order. Write a Python program to sort a list of strings (filenames) based on the length of each element.

```
filenames = input("Enter filenames separated by space: ").split()

filenames.sort(key=len)

print("Sorted by length:", filenames)
```

```
Enter filenames separated by space: AIRBAGS ballistics ai
Sorted by length: ['ai', 'AIRBAGS', 'ballistics']
```

2# A marketing team has two separate lists of email addresses from two different campaigns. Write a Python program to create a master list containing all unique emails from both lists (Union). (An email can be part of both lists)

```
list1 = set(input("Enter first email list: ").split())

list2 = set(input("Enter second email list: ").split())

master = list1.union(list2)

print("Master email list:", list(master))
```

```
Enter first email list: a@email b@yahoo
Enter second email list: f@gmail h@jingo
Master email list: ['h@jingo', 'a@email', 'b@yahoo', 'f@gmail']
```

3# A cyber-security tool is comparing two server access logs to identify suspicious IP addresses that appear in both logs. Write a Python program to find the common elements between two lists (Intersection).

```
list1 = set(input("Enter first list: ").split())

list2 = set(input("Enter second list: ").split())

common = list1.intersection(list2)

print("Common elements:", list(common))
```

```
Enter first list: 192.168.43.1 1001.1101.23
Enter second list: 192.168.43.1 76.89.001
Common elements: ['192.168.43.1']
```

4# A data visualization tool needs to plot a curve. Write a Python program that generates a list of tuples, where the first element is a number and the second is five times its cube, for a range of inputs.

```
start = int(input("Enter start value: "))
```

```
end = int(input("Enter end value: "))

result = []

for i in range(start, end+1):

    result.append((i, 5*(i**3)))


print(result)
```

```
Enter start value: 3
Enter end value: 5
[(3, 135), (4, 320), (5, 625)]
```

5# A Natural Language Processing (NLP) pre-processor needs to clean and analyze raw text data. Assume the variable data contains the string: "Different string Methods!". Write a Python script using string methods to: • Tokenize the string into a list of individual words. • Normalize the text by converting it to uppercase. • Find the index position of the substring "rules". • Search for a specific character (e.g., 'o') to see if it exists. • Sanitize the text by replacing the exclamation point with a question mark.

```
data = "Different string Methods!"


# tokenize

words = data.split()


# uppercase

upper_text = data.upper()


# find substring

index = data.find("rules")


# search character

exists = 'o' in data


# replace symbol

sanitized = data.replace("!", "?")


print("Tokens:", words)

print("Uppercase:", upper_text)
```

```
print("Index of 'rules':", index)

print("Contains 'o':", exists)

print("Sanitized:", sanitized)
```

```
Tokens: ['Different', 'string', 'Methods!']
Uppercase: DIFFERENT STRING METHODS!
Index of 'rules': -1
Contains 'o': True
Sanitized: Different string Methods?
```

6# A data compression utility needs to analyze the entropy of a text file. Write a Python program to calculate the frequency of every character present in a given string.

```
text = input("Enter a string: ")

freq = {}

for char in text:

    if char in freq:

        freq[char] += 1

    else:

        freq[char] = 1

print("Character frequencies:")

for k,v in freq.items():

    print(k, ":", v)
```

```
Enter a string: MISSISSIPPI
Character frequencies:
M : 1
I : 4
S : 4
P : 2
```

7# A simple encryption tool obfuscates passwords by reading them backward. Write a Python program to reverse the order of characters in a user-provided string. Example: • Input: "Security" • Output: "ytiruceS"

```
text = input("Enter string: ")

reverse = text[::-1]

print("Reversed:", reverse)
```

```
Enter string: aiden
Reversed: nedia
```

8# A product labeling system generates short "SKU codes" based on product names. Write a Python program that takes a string and generates a new string made of the first 2 and last 2

characters. (Constraint: If the product name is shorter than 2 characters, return an empty string) Example: • Input: "Panasonic" • Output: "Paic"

```
text = input("Enter product name: ")

if len(text) < 2:

    result = ""

else:

    result = text[:2] + text[-2:]


print("SKU:", result)
```

```
Enter product name: Panasonic
SKU: Paic
```

9# A library catalog system contains sorted Book IDs. To optimize lookup speeds, write a Python program to implement Binary Search using the divide and conquer approach to find a specific Book ID.

```
arr = list(map(int, input("Enter sorted Book IDs: ").split()))

target = int(input("Enter Book ID to search: "))

low = 0

high = len(arr) - 1

found = False

while low <= high:

    mid = (low + high) // 2


    if arr[mid] == target:

        print("Book ID found at index", mid)

        found = True

        break

    elif arr[mid] < target:

        low = mid + 1

    else:

        high = mid - 1


if not found:
```

```
    print("Book ID not found")
```

```
Enter sorted Book IDs: 1001 1002 2004 5001
Enter Book ID to search: 2004
Book ID found at index 2
```

10# An educational algorithm visualizer needs to show students how sorting works step-by-step. Write a Python program to implement the Insertion Sort algorithm that prints the state of the list after every iteration to visualize the intermediate steps.

```
arr = list(map(int, input("Enter numbers: ").split()))

for i in range(1, len(arr)):

    key = arr[i]

    j = i - 1

    while j >= 0 and key < arr[j]:

        arr[j+1] = arr[j]

        j -= 1


    arr[j+1] = key

    print("Step", i, ":", arr)
```

```
Enter numbers: 0 4 5 3 1 2
Step 1 : [0, 4, 5, 3, 1, 2]
Step 2 : [0, 4, 5, 3, 1, 2]
Step 3 : [0, 3, 4, 5, 1, 2]
Step 4 : [0, 1, 3, 4, 5, 2]
Step 5 : [0, 1, 2, 3, 4, 5]
```